

Redrob Intelligent Candidate Discovery & Ranking System

A Multi-Signal Hybrid Search, Verification, and Anti-Cheat Recruitment Pipeline

Author / Team: Hackathon Candidate Submission

Target Role: Senior AI Engineer

Technology Stack: Local Embeddings (all-MiniLM-L6-v2) + Multi-Signal Override & Caching Layer

Date: June 2026

1. Problem Framing & Recruiting Realities

Traditional candidate search and ranking systems are fundamentally broken. Standard platforms rely on simple keyword queries (e.g. matching 'RAG', 'Vector Database', or 'Python') to filter profiles. This rewards candidate profile manipulation (keyword stuffing) and creates high noise for recruiters, often filtering out high-quality applicants who describe their contributions using natural, plain language.

The Hackathon Challenge Constraints:

- **Volume & Noise:** Processing 100,000 candidate profiles where only a tiny fraction are genuinely relevant.
- **Rigid Schema Rules:** The system must strictly filter location (preference for Pune/Noida, hybrid compatibility) and target experience bands (5–9 years).
- **Designed Failure Modes (Traps):** The candidate pool contains specific target 'trap' profiles designed to break standard lexical matching models. Identifying and handling these traps is critical to ranking accuracy.
- **Operational Constraints:** The entire pipeline must run in under 90 seconds (excluding the one-time precomputation step) on CPU, with a maximum RAM footprint of 16 GB.

2. Hybrid Multi-Layer Architecture

To achieve both deep semantic understanding and strict adherence to rule-based criteria, the system uses a multi-layered hybrid architecture. It combines structured feature extraction, local vector embeddings, and rule-based overrides.

Pipeline Layer	Component & File	Logic & Description
1. JD Parsing	src/parse_jd.py	Extracts must-have skills, disqualifier flags, experience bounds, and target locations from the job description docx.
2. Structured Features	src/features.py	Extracts years of experience, current title match, product company exposure, Python relevance, and notice periods from candidate profile records.
3. Semantic Matching	src/embeddings.py	Generates dense sentence embeddings for candidate summary and history using local CPU-based all-MiniLM-L6-v2 model.
4. Overrides & Penalty	src/scoring.py	Applies multiplicative down-weights to services-only, research-only, keyword stuffers, and completely filters honeypots.
5. Behavioral Boost	src/features.py	Integrates platform engagement signals (notice period, response rates, last active, open to work) as a multiplier.

Performance Note: Features and embeddings are precomputed offline and cached to disk in artifacts/. During ranking, loading the cache and computing candidate scores takes less than **2.5 seconds** for the entire 100K candidate dataset.

3. Handling Intentionally Designed Data Traps

The evaluation candidate dataset features several engineered patterns that trick basic search queries. Below is how the scoring system handles each trap:

Designed Trap	System Handling & Logic	Fit Impact
Keyword Stuffers (e.g., Marketing Managers claiming AI skills with 0 months duration)	Checks current title against irrelevant titles. If the title is non-technical and AI skills are listed with 0 months experience, the flag is set.	Max-score capped at 0.12 (driving them out of the top shortlist)
Plain-Language Substance (e.g., True engineers who do not use popular buzzwords)	Evaluates synonyms (e.g., dense retrieval, approximate nearest neighbor) and relies on the semantic embedding layer to match intent rather than matching strings.	Highest score weight for actual retrieval and deployment history
Research-Only Profiles (e.g., Academic researchers without industry deployment)	Identifies university/lab experience. Cross-checks for lack of production metrics (SLA, latency, API, microservice, scale).	88% score deduction (filtered out from production roles)
Services-Only (e.g., TCS/Wipro career with zero product company exposure)	Checks career history for consultancy companies. Triggers if company matches list and services duration exceeds 60% with zero product background.	50% score deduction
Honeypot Profiles (e.g., single-job duration >35 years, future dates, mismatch in claimed dates)	Validates profile consistency. Checks for conflicting skill/career timeline metrics, future start dates, and single-job durations exceeding 35 years.	Score zeroed out (<0.5% effective score)

4. Key Design Decisions & Rationale

The scoring logic and technical constraints were built around realistic recruiter considerations. Here are the rationale behind the primary configurations:

- **Current Title Alignment (Weight: 0.22)**

With 100K profiles, roughly 55% are completely irrelevant (Accountants, HR, Civil Engineers). Title relevance serves as the primary filter to concentrate attention on engineering candidates before detailed text evaluation.

- **Multiplicative Behavioral Availability Multiplier (Range: 0.45x - 1.10x)**

Candidate response rates, notice periods, and active dates are combined and applied as a multiplier. This ensures that a highly aligned candidate who is active and available is ranked above an identical candidate who has a 90-day notice period and low platform engagement.

- **Location & Visa Filter via profile.country (Weight: 0.05 + Hard Penalties)**

Candidate location data is parsed directly from the nested profile.country sub-object. International applicants without relocation options are heavily down-weighted to minimize visa sponsorship risks, favoring local Pune/Noida (hybrid) candidates.

- **Balanced Semantic Weight (0.32 Semantic vs. 0.68 Structured)**

Relying too heavily on semantic match results in high scores for candidates who write good profile summaries but lack career substance. By keeping structured experience as the main driver, we prevent candidates from matching on summaries alone.

5. Limitations & Future Development Plan

While the pipeline is robust and passes all test constraints, the following enhancements are planned for future iterations:

Current Implementation Limitations:

1. **Heuristic Company Classification:** Services and academic institutions are detected using keyword lists. A services-company engineer who happens to work at a subsidiary might be misclassified.
2. **Outlier Honeypot Detection:** The validation of impossible career timelines uses statistical outlier rules (e.g. >35 years single tenure) rather than integrating a live corporate founding-date lookup.
3. **Standardized Reasoning:** The generated reasoning uses structured templates. While personalized with actual candidate facts (e.g. current title, company, years, locations), it lacks free-form narrative variation.

What We'd Implement With More Time:

- **Cross-Encoder Re-ranking:** Use a fine-tuned cross-encoder model to re-rank the top 500 candidates identified by the bi-encoder matching layer for even tighter semantic alignment.
- **Corporate Date API Integration:** Connect to structured databases to verify that career start dates do not precede company registration dates (highly robust honeypot check).
- **Recruiter Feedback Calibration:** Add an interactive UI where recruiters can accept/reject recommendations, feeding data back into a logistic regression or classification model to auto-calibrate feature weights.
- **Local LLM Narrative Generation:** Use a quantized local model (e.g. Llama 3) to generate custom human-like reasoning strings for candidate descriptions instead of template formats.